POLITECNICO
MILANO 1863

Initial Setup and Toolchain Selection for QoR Surrogate Modeling

Berk Yonder – 10994918

Advanced Computer Architectures

Prof. Christian Pilato

April 1, 2026

Contents

1 Introduction **2**

2 Selected Toolchain **2**

3 Environment Setup **3**

4 Benchmark Selection **3**

5 Initial Intermediate Metrics for QoR Prediction **4**

5.1 Scheduling Metrics 4

5.2 Resource Allocation Metrics 4

5.3 Area Estimation Metrics 4

6 Next Step: Automated Feature Extraction **4**

1 Introduction

The objective of this project is to develop a machine learning–based surrogate model capable of predicting the final Quality of Results (QoR) of hardware designs during the early stages of the ASIC compilation flow. Instead of executing the full synthesis and place-and-route pipeline for every design configuration, the goal is to estimate final metrics such as latency and area using information extracted from earlier stages of the High-Level Synthesis (HLS) process.

Modern HLS tools generate detailed intermediate reports containing scheduling information, resource allocation statistics, and early hardware estimations. These intermediate metrics provide valuable insights into the expected performance and resource utilization of the final hardware implementation.

The main idea of this project is therefore to collect these early-stage metrics, organize them into a structured dataset, and train regression models that can predict final QoR metrics. This approach can significantly reduce the need for expensive backend compilation runs and enable faster exploration of design configurations.

2 Selected Toolchain

The hardware synthesis flow used in this project is based on PandA-Bambu, an open-source High-Level Synthesis (HLS) framework developed at the Politecnico di Milano.

Bambu converts high-level C/C++ descriptions into hardware implementations through several compilation stages:

- frontend code analysis,
- intermediate representation generation,
- scheduling and resource allocation,
- hardware module binding,
- RTL hardware generation.

During these stages, Bambu produces detailed synthesis reports describing scheduling decisions, estimated latency, and resource utilization. These reports provide the intermediate metrics required for training surrogate QoR prediction models.

The experimental environment was configured as follows.

Component	**Description**
Operating System	Ubuntu 24.04 LTS (VirtualBox)
Host System	Windows
HLS Tool	PandA-Bambu
Programming Language	Python
Development Environment	Visual Studio Code

Table 1: Experimental environment configuration

Ubuntu was used to run the synthesis flow because the PandA-Bambu framework relies on several Linux-based compiler toolchains and build utilities.

3 Environment Setup

The development environment was deployed using Ubuntu 24.04 LTS running inside Oracle VirtualBox. This setup allows the Linux-based HLS toolchain to operate while development tools such as Visual Studio Code and Git are used on the host system.

The PandA-Bambu framework was cloned from the official repository:

https://github.com/ferrandi/PandA-bambu

The repository contains several submodules providing additional dependencies and benchmark collections. These submodules were initialized using the following command:

```
git submodule update --init --recursive
```

This step downloads auxiliary libraries and benchmark suites used by the framework, including:

- Abseil,
- pugixml,
- CHStone benchmark suite,
- MachSuite benchmark suite,
- PolyBench benchmark suite.

To compile Bambu successfully on Ubuntu 24.04, several development libraries and compiler toolchains were installed. Because the official documentation targets earlier Ubuntu versions, some package names required adaptation for modern distributions.

The required dependencies were installed using the following command:

```
sudo apt install -y \
autoconf autoconf-archive automake libtool pkg-config \
gcc-13 g++-13 gcc-13-plugin-dev gcc-13-multilib g++-13-multilib \
gfortran-13 gfortran-13-multilib \
clang-16 libclang-16-dev \
libbdd-dev libboost-all-dev libmpc-dev libmpfr-dev \
libxml2-dev liblzma-dev libmpfi-dev zlib1g-dev libicu-dev \
bison flex doxygen graphviz iverilog verilator make \
libsuitesparse-dev libglpk-dev
```

This command is presented as a code block because it is a shell command used directly in the terminal and should remain visually distinct from the main text in a technical report.

These dependencies support several internal components of the HLS compilation pipeline, including optimization routines, scheduling algorithms, parsing utilities, and compiler plugin infrastructure.

After installation and compilation, the Bambu executable was successfully built and validated by synthesizing a test benchmark.

4 Benchmark Selection

To generate training data for the QoR prediction model, this project uses benchmarks from the PolyBench suite included in the PandA-Bambu repository.

PolyBench is a widely used benchmark collection containing computational kernels imple-

mented in C. These kernels represent common numerical and scientific workloads and are frequently used for evaluating compiler optimizations and HLS tools.

Typical PolyBench kernels include:

- matrix multiplication (GEMM),
- linear algebra routines,
- stencil computations,
- convolution operations.

For the initial validation of the toolchain, the GEMM (General Matrix Multiplication) kernel was synthesized using PandA-Bambu. The synthesis successfully generated hardware reports and estimated QoR metrics.

5 Initial Intermediate Metrics for QoR Prediction

The central idea of this project is to extract intermediate metrics produced during the HLS compilation flow and use them as input features for machine learning models.

Based on the synthesis reports generated by PandA-Bambu, the following metrics were identified as potential predictors of final QoR.

5.1 Scheduling Metrics

- `control_steps` – number of scheduling cycles,
- `min_slack` – minimum timing slack,
- `frequency_mhz` – estimated maximum clock frequency,
- `states` – number of finite state machine states.

5.2 Resource Allocation Metrics

- `modules_instantiated` – number of hardware modules,
- `performance_conflicts` – detected resource conflicts,
- `registers` – allocated registers,
- `flipflops` – total number of flip-flops.

5.3 Area Estimation Metrics

- `area_est` – estimated logic resource area,
- `mux_area` – estimated multiplexer area,
- `total_area` – total estimated hardware area,
- `dsps` – number of DSP blocks.

These metrics are extracted directly from the synthesis logs produced by the Bambu toolchain. Each synthesis run will produce a set of feature values corresponding to a benchmark configuration.

6 Next Step: Automated Feature Extraction

The next stage of the project will focus on automatically extracting these metrics from Bambu synthesis logs.

A Python-based parser will be implemented to read synthesis reports and convert the extracted metrics into a structured dataset. Each row of the dataset will correspond to a synthesis run of a benchmark kernel.

The dataset will then be used to train regression models capable of predicting QoR metrics such as latency and hardware area from early HLS outputs.

This dataset will serve as the foundation for building surrogate models that enable faster design space exploration in hardware synthesis workflows.